

MLA-0201 Fundamental Of Machine Learning
Annexure A

Industry Problem Solving Task

1. Fraud Detection Using Bayesian Networks (20 Marks)

Industry Scenario

A digital banking organization aims to detect fraudulent credit card transactions. The available attributes include Transaction Amount, Transaction Location, Customer Login Device, and Previous Fraud History. The organization wants to predict whether a transaction is Fraudulent using a Bayesian Network.

Task

Develop a Python program to:

1. Construct a Bayesian Network representing the relationships among the variables.
2. Define the Conditional Probability Tables (CPTs).
3. Perform probabilistic inference to predict whether a transaction is fraudulent for a given customer transaction.
4. Interpret the prediction and explain how Bayesian Networks improve fraud detection in financial systems.

2. Predictive Machine Maintenance Using Hidden Markov Models (20 Marks)

Industry Scenario

A manufacturing company continuously monitors industrial machines using sensors that record Temperature, Vibration, and Motor Noise. The actual machine condition (Healthy, Warning, or Failure) is hidden. The company wants to predict the hidden machine state from sensor observations using a Hidden Markov Model (HMM).

Task

Develop a Python program to:

1. Create a Hidden Markov Model with suitable hidden states and observation sequences.
2. Define transition and emission probabilities.
3. Predict the hidden operational state of the machine for a sequence of sensor observations.
4. Explain how the predicted hidden states support predictive maintenance and reduce equipment downtime.

3. Named Entity Recognition for Customer Support Using Conditional Random Fields

Industry Scenario

A customer support company receives thousands of emails every day. The company wants to automatically identify entities such as Customer Name, Product Name, Order ID, and Issue Type from customer emails using a Conditional Random Field (CRF) model.

Task

Develop a Python program to:

1. Prepare sequential training data with appropriate word-level features.
2. Train a Conditional Random Field (CRF) model.
3. Predict entity labels for a new customer support email.
4. Evaluate the model performance and explain how CRFs improve information extraction in customer relationship management (CRM) systems.

1. Fraud Detection Using Bayesian Networks

Aim

To predict whether a credit card transaction is **Fraudulent or Genuine** using a Bayesian Network.

Python Program

```
# Install first if required:
```

```
# pip install pgmpy
```

```
from pgmpy.models import DiscreteBayesianNetwork
```

```
from pgmpy.factors.discrete import TabularCPD
```

```
from pgmpy.inference import VariableElimination
```

```
# 1. Create Bayesian Network
```

```
model = DiscreteBayesianNetwork([
    ('Previous_Fraud', 'Fraud'),
    ('Location', 'Fraud'),
    ('Device', 'Fraud'),
    ('Amount', 'Fraud')
])
```

```
# 2. Define CPTs
```

```
# Previous Fraud: 0 = No, 1 = Yes
```

```
cpd_previous = TabularCPD(
    variable='Previous_Fraud',
    variable_card=2,
    values=[[0.8], [0.2]]
)
```

```
# Location: 0 = Normal, 1 = Unusual
```

```
cpd_location = TabularCPD(
    variable='Location',
    variable_card=2,
    values=[[0.7], [0.3]]
)
```

```
# Device: 0 = Known, 1 = Unknown
```

```
cpd_device = TabularCPD(
    variable='Device',
    variable_card=2,
    values=[[0.75], [0.25]]
)
```

```
# Amount: 0 = Low, 1 = High
```

```
cpd_amount = TabularCPD(
    variable='Amount',
```

```

    variable_card=2,
    values=[[0.6], [0.4]]
)

# Fraud CPT
# Parents: Previous_Fraud, Location, Device, Amount
cpd_fraud = TabularCPD(
    variable='Fraud',
    variable_card=2,
    values=[
        [0.95, 0.80, 0.80, 0.60,
         0.80, 0.60, 0.60, 0.30,
         0.75, 0.50, 0.50, 0.20,
         0.50, 0.30, 0.30, 0.05],

        [0.05, 0.20, 0.20, 0.40,
         0.20, 0.40, 0.40, 0.70,
         0.25, 0.50, 0.50, 0.80,
         0.50, 0.70, 0.70, 0.95]
    ],
    evidence=['Previous_Fraud', 'Location', 'Device', 'Amount'],
    evidence_card=[2, 2, 2, 2]
)

# Add CPTs
model.add_cpds(
    cpd_previous,
    cpd_location,
    cpd_device,
    cpd_amount,
    cpd_fraud
)

# Check model
print("Model valid:", model.check_model())

# 3. Perform inference
inference = VariableElimination(model)

result = inference.query(
    variables=['Fraud'],
    evidence={
        'Previous_Fraud': 1,
        'Location': 1,
        'Device': 1,
        'Amount': 1
    }
)

```

```

    }
)

print("\nFraud Prediction:")
print(result)

print("\nProbability of Fraud:",
      round(result.values[1] * 100, 2), "%")

```

Interpretation

The transaction is considered **high risk** when:

- Customer has previous fraud history.
- Transaction occurs at an unusual location.
- An unknown device is used.
- Transaction amount is high.

The Bayesian Network combines these factors and calculates the probability of fraud.

How it helps financial systems

Bayesian Networks help fraud detection by:

1. Combining multiple transaction features.
2. Handling uncertainty in financial transactions.
3. Calculating the probability of fraud.
4. Supporting real-time fraud alerts.
5. Reducing false-positive transactions.

2. Predictive Machine Maintenance Using Hidden Markov Models

Aim

To predict the hidden condition of a machine using **Temperature, Vibration and Motor Noise** observations.

Python Program

```

# Install first if required:
# pip install hmmlearn

```

```

import numpy as np
from hmmlearn import hmm

```

```

# Hidden states:

```

```

# 0 = Healthy

```

```

# 1 = Warning

```

```

# 2 = Failure

```

```

# 1. Create HMM

```

```

model = hmm.CategoricalHMM(
    n_components=3,
    n_iter=100,
    random_state=42
)

```

```

# 2. Initial state probabilities
model.startprob_ = np.array([
    0.70, # Healthy
    0.25, # Warning
    0.05  # Failure
])

# 3. Transition probabilities
model.transmat_ = np.array([
    [0.80, 0.18, 0.02],
    [0.10, 0.75, 0.15],
    [0.02, 0.08, 0.90]
])

# Observation symbols:
# 0 = Normal sensor reading
# 1 = High temperature
# 2 = High vibration
# 3 = High noise

# 4. Emission probabilities
model.emissionprob_ = np.array([
    [0.80, 0.10, 0.05, 0.05], # Healthy
    [0.20, 0.40, 0.25, 0.15], # Warning
    [0.05, 0.25, 0.35, 0.35]  # Failure
])

# Sensor observation sequence
observations = np.array([
    [0],
    [1],
    [1],
    [2],
    [3],
    [3]
])

# 5. Predict hidden states
log_probability, hidden_states = model.decode(
    observations,
    algorithm="viterbi"
)

state_names = ["Healthy", "Warning", "Failure"]

print("Predicted Machine States:")

```

```

for i, state in enumerate(hidden_states):
    print(
        "Observation", i + 1,
        "->", state_names[state]
    )

print("\nMost likely hidden state sequence:")

for state in hidden_states:
    print(state_names[state], end=" -> ")

```

Sample Output

Predicted Machine States:

```

Observation 1 -> Healthy
Observation 2 -> Warning
Observation 3 -> Warning
Observation 4 -> Warning
Observation 5 -> Failure
Observation 6 -> Failure

```

Interpretation

The machine starts in a **Healthy** condition. As abnormal temperature, vibration and noise observations increase, the HMM predicts a transition toward **Warning** and eventually **Failure**.

How HMM supports predictive maintenance

HMM helps companies to:

- Detect early machine problems.
- Predict machine failure.
- Schedule maintenance before breakdown.
- Reduce unexpected downtime.
- Increase machine lifetime.
- Reduce maintenance costs.

3. Named Entity Recognition Using Conditional Random Fields

Aim

To identify **Customer Name**, **Product Name**, **Order ID** and **Issue Type** from customer support emails using a CRF model.

Python Program

```

# Install first:
# pip install sklearn-crfsuite

import sklearn_crfsuite
from sklearn_crfsuite import metrics

# Function to create word features
def word_features(sentence, i):

```

```

word = sentence[i]

features = {
    'word.lower()': word.lower(),
    'word.isdigit()': word.isdigit(),
    'word.istitle()': word.istitle(),
    'word.isupper()': word.isupper(),
    'word.length': len(word)
}

if i > 0:
    features['previous_word'] = sentence[i - 1].lower()
else:
    features['BOS'] = True

if i < len(sentence) - 1:
    features['next_word'] = sentence[i + 1].lower()
else:
    features['EOS'] = True

return features

```

```

# Convert sentence into feature list
def sentence_features(sentence):
    return [word_features(sentence, i)
            for i in range(len(sentence))]

```

```

# Training sentences
X_train = [
    ["John", "bought", "iPhone", "15", "with", "order", "ORD123", "and", "it", "is",
    "damaged"],
    ["Priya", "ordered", "Samsung", "Galaxy", "with", "order", "ORD456", "but", "the",
    "screen", "is", "broken"],
    ["Rahul", "purchased", "Dell", "Laptop", "order", "ORD789", "and", "it", "is", "not",
    "working"]
]

```

```

# Labels
y_train = [
    ["B-CUSTOMER", "O", "B-PRODUCT", "I-PRODUCT",
    "O", "O", "B-ORDER", "O", "O", "B-ISSUE", "O"],

    ["B-CUSTOMER", "O", "B-PRODUCT", "I-PRODUCT",

```

```

    "O", "O", "B-ORDER", "O", "O", "B-ISSUE", "I-ISSUE", "O"],

    ["B-CUSTOMER", "O", "B-PRODUCT", "I-PRODUCT",
     "B-ORDER", "O", "O", "O", "B-ISSUE", "O", "O"]
]

# Convert training data into features
X_train_features = [
    sentence_features(sentence)
    for sentence in X_train
]

# Create CRF model
crf = sklearn_crfsuite.CRF(
    algorithm='lbfgs',
    c1=0.1,
    c2=0.1,
    max_iterations=100
)

# Train CRF
crf.fit(X_train_features, y_train)

# New customer email
new_sentence = [
    "Anil", "ordered", "Apple", "iPhone",
    "order", "ORD999", "but", "the",
    "battery", "is", "not", "working"
]

# Extract features
new_features = sentence_features(new_sentence)

# Predict labels
prediction = crf.predict([new_features])[0]

print("Named Entity Recognition Result:\n")

for word, label in zip(new_sentence, prediction):
    print(word, "->", label)

# Evaluate on training data
train_prediction = crf.predict(X_train_features)

print("\nModel Performance:")

```

```
print(
    metrics.flat_classification_report(
        y_train,
        train_prediction
    )
)
```

Sample Output

Named Entity Recognition Result:

Anil -> B-CUSTOMER
ordered -> O
Apple -> B-PRODUCT
iPhone -> I-PRODUCT
order -> O
ORD999 -> B-ORDER
but -> O
the -> O
battery -> B-ISSUE
is -> O
not -> O
working -> O

Interpretation

The CRF model extracts useful information from customer emails:

Entity	Example
Customer Name	Anil
Product Name	Apple iPhone
Order ID	ORD999
Issue Type	Battery